DocuVault AI Assistant

Technical Documentation

Chatbot & Voice Bot — Architecture, Implementation & Configuration

Property	Details
Version	2.0
Date	March 2024
Stack	Django 4 · ChromaDB · Groq (LLaMA-3) · Web Speech API · SSE
Scope	Voice Bot · Text Chatbot · RAG Pipeline · TTS/STT

Table of Contents

SECTION 1

System Architecture Overview

DocuVault AI Assistant is a full-stack Django web application combining a document management system with an AI-powered chatbot. The system uses Retrieval-Augmented Generation (RAG) to ground AI responses in uploaded documents. Voice input and output are handled entirely client-side using the Web Speech API, with a server-side Whisper fallback for unsupported browsers.

High-Level Architecture

Layer	Component	Technology
Frontend	Chat UI + Voice Modal	HTML/CSS/Vanilla JS
Frontend	Speech-to-Text (STT)	Web Speech API (SpeechRecognition)
Frontend	Text-to-Speech (TTS)	Web Speech Synthesis API
Frontend	Streaming client	fetch() + ReadableStream + SSE parser
Backend	Web framework	Django 4.x
Backend	LLM inference	Groq API via LangChain ChatGroq
Backend	LLM model	LLaMA-3 70B (groq/llama3-70b-8192)
Backend	Vector store	ChromaDB (local persistence)
Backend	Embeddings	HuggingFace sentence-transformers
Backend	PDF parsing	PyMuPDF (fitz) + pdfplumber
Backend	STT fallback	faster-whisper (server-side)
Backend	Streaming protocol	Server-Sent Events (SSE) over HTTP

Request Flow

1. User types or speaks a question.
2. If voice: SpeechRecognition converts audio to text (client-side).
3. Text is POST-ed to /chatbot/query/stream/ with CSRF token.
4. Django retrieves relevant document chunks from ChromaDB (RAG).
5. Chunks + conversation history are passed to Groq LLM as context.
6. LLM streams tokens back via SSE (Server-Sent Events).
7. Frontend renders tokens progressively into the chat bubble.
8. TTS speaks each sentence as it arrives (voice mode only).
9. After response completes, voice mode auto-resumes listening.

SECTION 2

RAG Pipeline — Retrieval-Augmented Generation

RAG allows the chatbot to answer questions grounded in your uploaded documents rather than relying solely on the LLM's training data. Documents are chunked, embedded into vectors, and stored in ChromaDB. At query time, the most relevant chunks are retrieved and injected into the LLM prompt.

Auto-Indexing Pipeline

Documents are automatically indexed when uploaded. No manual step required.

1. User uploads a PDF through the DocuVault interface.
2. Django signals.py fires post_save on the Document model.
3. A daemon background thread calls _index_in_background(document_id).
4. PDF is parsed page-by-page using PyMuPDF / pdfplumber.
5. Text is split into overlapping chunks (512 tokens, 50-token overlap).
6. Each chunk is embedded using HuggingFace sentence-transformers.
7. Embeddings are stored in ChromaDB with metadata (doc title, page number).
8. DocumentEmbedding model updated: is_indexed=True, status=completed.

Hybrid Search

At query time, the system performs hybrid retrieval combining semantic similarity (cosine distance on embeddings) with keyword matching (BM25-style). Results are re-ranked and the top-N chunks are selected as context.

Source Attribution

Each retrieved chunk carries metadata: document title, page number, and content type (text, table, image). After the LLM responds, the frontend displays source chips below the answer showing which document pages were used. If no document context was used, a "General knowledge" badge is shown instead.

Key Files

File	Purpose
documents/signals.py	Django post_save signal — triggers auto-indexing
documents/rag/llm_manager.py	Groq LLM wrapper with stream() support
documents/rag/conversation.py	query_stream() — RAG retrieval + LLM call
documents/rag/config.py	SYSTEM_PROMPT — document-first instructions
documents/rag_views.py	chatbot_query_stream_view — SSE endpoint

SECTION 3

Text Chatbot — Implementation

The text chatbot is a streaming chat interface built with vanilla JavaScript. Messages are rendered progressively as tokens arrive from the server. Markdown formatting (bold, italic, code, lists) is applied in real time.

Frontend — sendQuery() Function

Core function that handles the full request-response cycle:

```
async function sendQuery(override) {
// 1. Cancel any in-progress stream (AbortController)
if (streamAbort) { streamAbort.abort(); streamAbort = null; }
streamAbort = new AbortController();
setBusy(true);
addMsg("human", esc(q), []); // show user bubble
const bub = mkBub(); // create empty AI bubble
// 2. POST to SSE endpoint
const resp = await fetch("/chatbot/query/stream/", {
method: "POST", body: fd,
signal: streamAbort.signal // allows cancellation
});
// 3. Read SSE stream token by token
const reader = resp.body.getReader();
while (true) {
const { done, value } = await reader.read();
if (done) break;
// parse SSE events: session, sources, token, done, error
}
}
```

SSE Event Types

Event Type	Payload	Action
session	{ session_id: 123 }	Updates sid variable for conversation continuity
sources	{ data: [...] }	Renders source chips below AI bubble immediately
token	{ data: "word " }	Appends token to bubble, triggers TTS sentence flush
done	{ data: "full text" }	Finalises bubble, saves to DB, restarts voice if active
error	{ data: "msg" }	Shows error message in bubble

SECTION 4

SSE Streaming — How Real-Time Responses Work

Server-Sent Events (SSE) allow the server to push data to the browser progressively over a single HTTP connection. This enables the chatbot to display text and speak audio before the full AI response is complete, giving a natural real-time feel.

Why SSE Instead of WebSockets?

- SSE works over standard HTTP/HTTPS — no special server configuration needed.
- SSE is one-directional (server to client) which matches the chat pattern perfectly.
- Built-in reconnection and event ID support in the browser.
- EventSource API is native — however, since we use POST (not GET), we use fetch() + ReadableStream instead of EventSource directly.

Django Backend — chatbot_query_stream_view

```
@login_required
@require_http_methods(["POST"])
def chatbot_query_stream_view(request):
def sse_generator():
# 1. Emit session ID
yield f'data: {json.dumps({"type":"session","session_id":...})}\n\n'
for event in chatbot.query_stream(question=question):
if event["type"] == "sources":
yield f'data: {json.dumps({"type":"sources","data":...})}\n\n'
elif event["type"] == "token":
yield f'data: {json.dumps({"type":"token","data":chunk})}\n\n'
elif event["type"] == "done":
yield f'data: {json.dumps({"type":"done","data":...})}\n\n'
response = StreamingHttpResponse(sse_generator(),
content_type="text/event-stream")
response["Cache-Control"] = "no-cache"
response["X-Accel-Buffering"] = "no" # disables nginx buffering
return response
```

AbortController — Cancelling Stale Streams

When a new query is sent while a previous stream is still reading, the old fetch is cancelled via AbortController. This prevents stacked network connections and duplicate bubble rendering.

```
let streamAbort = null;
async function sendQuery(override) {
if (streamAbort) { streamAbort.abort(); } // cancel previous
streamAbort = new AbortController();
const resp = await fetch(url, { signal: streamAbort.signal, ... });
// ...
// In finally block:
streamAbort = null;
```

```
}
```

SECTION 5

Voice Bot — Speech-to-Text (STT)

Speech recognition is handled client-side using the Web Speech API (SpeechRecognition). This runs entirely in the browser — no audio is sent to the Django server for transcription unless the browser does not support the API, in which case a server-side Whisper fallback is used.

SpeechRecognition Setup

```
const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
recog = new SR();
recog.continuous = true; // keeps listening until stopped
recog.interimResults = true; // shows partial results in real time
recog.lang = "en-IN"; // accent / language for recognition
recog.maxAlternatives = 1;
```

Key Events

Event	What It Does
onstart	Sets UI to Listening state, updates status text
onresult	Accumulates final + interim transcript; resets 2-second silence timer
onerror	Handles not-allowed (mic blocked), no-speech (silence), language-not-supported (falls back to en-US)
onend	Fires when recognition stops; triggers tryRst() to restart if still in voice session

Silence Detection — Auto-Send

A 2-second silence timer is reset on every onresult event. When the timer fires (user stops speaking for 2 seconds), the accumulated transcript is automatically sent as a query without any button press.

```
recog.onresult = (e) => {
// ... accumulate transcript ...
clearTimeout(silTimer);
silTimer = setTimeout(() => {
const final = accumulated.trim();
if (isListening && final) {
isListening = false;
_killRecog(); // null callbacks, then abort — no ghost events
sendQuery(final); // fire the query
}
}, 2000); // 2 second silence threshold
};
```

_killRecog() — Preventing Memory Leaks

A critical helper that nullifies ALL event callbacks before calling abort(). Without this, Chrome fires ghost onend events after abort(), which triggers tryRst() creating a new SR instance — causing memory leaks that crash the tab after 2-3 voice queries.

```
function _killRecog() {
if (recog) {
recog.onresult = null; // prevent ghost events
recog.onerror = null;
recog.onend = null;
recog.onstart = null;
try { recog.abort(); } catch {}
recog = null;
}
}
```

Server-Side Whisper Fallback

If the browser does not support SpeechRecognition, audio is recorded via MediaRecorder and uploaded to /chatbot/voice/transcribe/ where faster-whisper transcribes it server-side.

Install: pip install faster-whisper

SECTION 6

Voice Bot — Text-to-Speech (TTS)

AI responses are spoken aloud using the Web Speech Synthesis API. TTS is activated in two scenarios: (1) when the user toggles Voice On via the toolbar button, or (2) automatically when the user is in a voice session (used the mic button).

Voice Selection Priority

The system tries to find the best available voice on the user's device in this order:

```
const want = ["Microsoft Heera", // Indian English female (Windows)
"Microsoft Ravi", // Indian English male (Windows)
"Lekha", // Indian English (macOS)
"Veena"]; // Indian English (macOS)
// Fallback chain:
bestV = vs.find(v => v.lang === "en-IN") // any en-IN voice
|| vs.find(v => v.lang.startsWith("en")) // any English voice
|| vs[0]; // first available voice
```

Change Voice / Accent

To use a different accent, edit two locations in chatbot.html:

1. Recognition language (what it listens in):

```
let srLang = "en-IN"; // change to en-US, en-GB, en-AU etc.
```

2. Voice priority list (what it speaks in):

```
const want = ["Microsoft Heera", "Microsoft Ravi", ...];
```

To find available voices on your system, run in browser console:

```
speechSynthesis.getVoices().forEach(v => console.log(v.name, v.lang));
```

Speed and Pitch

```
utt.rate = 1.18; // speed: 1.0=normal, 1.5=fast, 2.0=max
utt.pitch = 1.05; // pitch: 0.5=low, 1.0=normal, 2.0=high
```

Both values are found in the pumpTTS() function in chatbot.html.

Sentence Streaming — TTS Starts Before Full Response

Rather than waiting for the complete AI response, TTS starts as soon as the first complete sentence arrives during streaming. A regex extracts sentences ending in . ! ? or newline and speaks them immediately.

```
function ttsFlush(buf) {
const re = /[^.!?\n]+[.!?\n]+/g;
let last = 0, m;
while ((m = re.exec(buf)) !== null) {
speakChunk(m[0].trim()); // speak each complete sentence
```

```
last = m.index + m[0].length;
}
return buf.slice(last); // return remaining partial sentence
}
```

TTS Queue Management

Issue	Solution
Chrome stops speaking silently	25-second safety timer in pumpTTS() recovers automatically
Queue overflow / memory spiral	Queue capped at 8 items; trimmed to 4 when exceeded
Corrupted queue from cancel()	speechSynthesis.cancel() removed from pumpTTS(); only called in stopSpeak()
Voice not loaded yet	loadVoices() called on voiceschanged event + 1s timeout fallback

SECTION 7

Voice Modal — UI States and Flow

When the mic button is clicked, a full-screen voice modal opens with an animated orb that changes colour based on the current voice state. The modal wraps the same underlying voice engine — it is purely a UI layer on top of the STT/TTS functions.

Modal States

State	Orb Colour	Visual	Description
idle	Purple	Slow pulse	Modal just opened, waiting to start
listening	Red	Fast pulse	Microphone active, user is speaking
processing	Amber	Spinning	Query sent, waiting for LLM response
speaking	Green	Wave bars animate	AI is speaking the response via TTS

State Transition Flow

openVoiceModal() -> idle state -> startVoice() after 400ms

startVoice() -> getUserMedia() permission check -> startSR()

startSR() -> listening state -> user speaks

silence 2s -> _killRecog() -> processing state -> sendQuery()

sendQuery() -> SSE stream -> tokens arrive -> pumpTTS()

pumpTTS() -> speaking state -> SpeechSynthesis speaks

TTS ends -> afterSpeak() -> scheduleNextListen() -> startSR()

-> back to listening state -> loop continues

Conversation Loop

After the AI finishes speaking, the system automatically returns to listening mode after 700ms. This creates a continuous voice conversation without the user needing to press any button. The loop continues until the user clicks Stop or closes the modal.

Modal Controls

Control	Action
Stop button (square icon)	If listening: stops and restarts. If speaking: stops TTS. If idle: closes modal.
Send Now button	Manually sends current transcript without waiting for silence timeout.
Close (x) button	Stops voice session, closes modal, returns focus to text input.
ESC key	Closes modal (same as Close button).
Click outside modal	Closes modal (click on dark overlay).

SECTION 8

Configuration Reference

Voice Settings — chatbot.html

Setting	Location	Default	Description
srLang	Line ~690	en-IN	Recognition language/accent for STT
utt.rate	pumpTTS()	1.18	TTS speech speed (0.1 to 2.0)
utt.pitch	pumpTTS()	1.05	TTS voice pitch (0.5 to 2.0)
want[]	loadVoices()	Microsoft Heera...	Ordered list of preferred TTS voices
2000 (ms)	silTimer	2000	Silence duration before auto-send (milliseconds)
700 (ms)	scheduleNextListen()	700	Delay before resuming listen after TTS ends
8 (items)	speakChunk()	8	Maximum TTS queue size before trimming

RAG Settings — documents/rag/config.py

Setting	Description
SYSTEM_PROMPT	Instructs LLM to prefer document context over general knowledge
n_results	Number of chunks retrieved per query (default: 5)
chunk_size	Token size per chunk when indexing PDFs (default: 512)
chunk_overlap	Overlap between adjacent chunks (default: 50 tokens)
use_hybrid	Enable hybrid semantic + keyword search (default: True)
use_rewrite	Enable query rewriting for better retrieval (default: True)

Django URLs — documents/urls.py

URL	View	Purpose
/chatbot/	chatbot_view	Main chatbot page (GET)
/chatbot/query/stream/	chatbot_query_stream_view	SSE streaming endpoint (POST)
/chatbot/voice/transcribe/	voice_transcribe_view	Whisper STT fallback (POST)
/chatbot/clear/	clear_chat	Clear conversation history (POST)

SECTION 9

Browser Compatibility

Feature	Chrome	Edge	Firefox	Safari
Text Chatbot (SSE)	Full	Full	Full	Full
SpeechRecognition (STT)	Full	Full	Not supported	Partial
SpeechSynthesis (TTS)	Full	Full	Full	Full
Indian voices (en-IN)	Yes (Windows)	Yes (Windows)	System only	macOS only
Voice Modal	Full	Full	No STT	Partial
Whisper Fallback (STT)	N/A	N/A	Yes	Yes

Recommended: Google Chrome or Microsoft Edge on Windows for full Indian English voice support (Microsoft Heera / Microsoft Ravi voices). Firefox users will use the server-side Whisper fallback for STT and system voices for TTS.

HTTPS Requirement

SpeechRecognition requires either HTTPS or localhost. On HTTP with a non-localhost hostname (e.g., http://192.168.1.10:8000), the browser will block microphone access and the voice bot will show a "Microphone blocked" error. For local development, always use http://127.0.0.1:8000 or http://localhost:8000. For production, deploy with HTTPS.

SECTION 10

Troubleshooting Guide

Voice bot stops working after 2-3 queries

Cause: Ghost onend events from un-cleaned SpeechRecognition instances stack up in memory.

Fixed by: _killRecog() which nulls all callbacks before abort().

If it still happens: refresh the page to reset all SR instances.

Check: Open DevTools > Memory tab > take heap snapshot to confirm no SR leak.

Microphone blocked / not-allowed error

Cause: Browser denied microphone permission.

Fix: Click the lock icon in Chrome address bar > Allow microphone.

Also ensure the page is served over HTTPS or localhost.

Fix: Go to Chrome Settings > Privacy > Site Settings > Microphone > Allow.

Voice bot shows Listening but no transcript appears

Cause: en-IN locale not supported on this system.

Fix: The system auto-falls back to en-US on language-not-supported error.

Alternative: Change srLang to "en-US" manually in chatbot.html line ~690.

Also check: Is there background noise? SpeechRecognition needs a clear audio signal.

TTS not speaking / AI is silent

Cause 1: No voices loaded yet. Voices load asynchronously on page load.

Fix: Click the Voice On button, wait 2 seconds, then try again.

Cause 2: speechSynthesis is paused (Chrome bug after tab switch).

Fix: Call speechSynthesis.resume() in browser console.

Cause 3: ttsOn is false and voiceActive is false (not in voice session).

Fix: Either toggle Voice On button or use the mic button to enter voice mode.

Browser tab crashes during voice session

Cause: Multiple SpeechRecognition instances running simultaneously (memory leak).

Fixed by: _killRecog() always destroying old instance before new SR() call.

Fixed by: TTS queue capped at 8 items to prevent memory spiral.

Fixed by: AbortController cancels stale SSE streams before new query.

AI ignores document content and gives general answers

Cause: Document not indexed yet (embedding still in processing state).

Check: Go to document list > check indexing status badge.

Fix: Wait for indexing to complete (background thread, usually under 30 seconds).

Cause 2: Query is too different from document content for RAG to retrieve relevant chunks.

Fix: Ask a more specific question using keywords from the document.

Responses are very slow

Cause: Groq API rate limit or network latency.

Check: View Django server console for API error messages.

Fix: Groq free tier has rate limits — upgrade plan or add retry logic.

Also: Large documents with many chunks take longer to search. Reduce n_results in config.py.

Source chips not appearing below answers

Cause: Query answered from general knowledge (no document chunks retrieved).

Expected behaviour: General knowledge badge shown instead.

If document was uploaded: Check is_indexed status in DocumentEmbedding model.

Fix: Re-upload the document or trigger re-indexing via admin panel.
